AWS MASTER NOTES

Concepts • Console Practicals • Projects • Revision

Purpose	Understand each service in simple words and use the notes to rebuild the practical projects.
Writing style	Short definitions, numbered steps, essential commands, flows and troubleshooting.
Important	Replace placeholders in commands with your own account ID, IP, resource names and Region. Verify current AWS console options.

Notebook rule: Meaning → Why → How it works → Practical → Verification.

Designed as a clean reference PDF for copying into an A4 notebook. Completed practicals and theory-only topics are clearly distinguished.

CONTENTS & LEARNING FLOW

• 1. AWS fundamentals

• 2. IAM — identity and permissions

• 3. EC2 — virtual servers

• 4. EC2 Hello World practical

• 5. S3 — object storage

• 6. S3 static website practical

• 7. VPC — 11 core topics and network flows

• 8. Docker and ECR

• 9. ECS and Fargate

• 10. Docker → ECR → ECS project

• 11. Lambda

• 12. API Gateway

• 13. S3 + Lambda CSV-cleaner project

• 14. ELB and ALB

• 15. Three-EC2 ALB project

• 16. Auto Scaling Group

• 17. ASG + Load Balancer project

• 18. CloudWatch and SNS

• 19. CPU alarm + email project

• 20. Syllabus extras: EKS and AWS CI/CD

• 21. Troubleshooting, cleanup and final revision

AWS basics → IAM → EC2 → S3 → VPC → Docker/ECR/ECS → Lambda/API Gateway → ALB/ASG → CloudWatch/SNS

The sequence follows the learning order and project list shared in this conversation. EKS and AWS-native CI/CD are theory/future topics unless separately completed.

1. AWS FUNDAMENTALS

1.1 Cloud computing

Meaning: Delivery of computing resources (servers, storage, databases and networking) over the internet.

Benefits: No upfront hardware purchase • flexible capacity • pay for usage • faster setup.

Deployment types: Public cloud, private cloud, hybrid cloud.

Service models: IaaS = infrastructure; PaaS = development platform; SaaS = ready-to-use software.

1.2 AWS and global infrastructure

AWS: Amazon Web Services, a cloud platform with compute, storage, networking, databases and security services.

Region: Geographic area where AWS resources are hosted. **Availability Zone (AZ):** Isolated infrastructure location within a Region.

Scalability: Adjust capacity as demand changes. **High availability:** Design to keep service running despite component failure.

1.3 Main service categories

Category	Examples / purpose
Compute	EC2 (servers), Lambda (serverless code)
Storage	S3 (objects), EBS (block storage)
Networking	VPC, Route 53
Containers	ECR (images), ECS/EKS (orchestration)
Monitoring	CloudWatch (metrics, logs, alarms)
Security	IAM (identity and permissions)
Integration	SNS (notifications), API Gateway (APIs)

Memory: EC2 runs servers • S3 stores objects • IAM controls access • VPC provides networking • CloudWatch monitors.

2. IAM — IDENTITY AND ACCESS MANAGEMENT

2.1 Main components

User: Identity for a person/application. **Group:** Collection of users. **Policy:** JSON permissions document. **Role:** Assumable identity that grants temporary permissions to a user/service.

2.2 Policies and permissions

Policy fields: **Effect** (Allow/Deny), **Action** (operation), **Resource** (target), optional **Condition**.

• Least privilege: give only the permissions required.

• An applicable explicit Deny overrides an Allow.

• IAM policy controls permissions; a resource policy can also grant access to a resource.

2.3 MFA and access keys

• MFA adds a second verification factor to sign-in.

• Access key ID + secret access key are for programmatic access; never share or hardcode the secret.

• EC2 SSH key pairs are different from IAM access keys.

2.4 AWS CLI practical

AWS CLI manages AWS resources from a terminal. Use a profile/role/credential method and the correct Region.

```
aws configure
aws sts get-caller-identity
aws ec2 describe-instances --region us-east-1
```

Verification: get-caller-identity confirms the identity being used. AccessDenied usually means missing permissions; never paste secrets into notes.

3. EC2 — VIRTUAL SERVERS

3.1 EC2, AMI and instance type

EC2: Virtual server in AWS. **AMI:** Image/template used to launch an instance. **Instance type:** CPU/memory/network capacity. **EBS:** Block storage volume. **User Data:** Startup script.

3.2 Key pairs, PEM/PPK, SSH/PuTTY

SSH: Encrypted remote terminal. **PEM:** Common private-key format. **PPK:** PuTTY private-key format. Use PuTTYgen to convert when needed.

Steps: launch instance with key pair → allow SSH (22) from trusted IP → use correct OS username → connect using SSH/PuTTY.

```
ssh -i key.pem ec2-user@PUBLIC-IP
```

Common usernames vary by AMI (for example, Amazon Linux often uses ec2-user; Ubuntu often uses ubuntu).

3.3 Security Groups and ports

Port	Use
22	SSH
80	HTTP
443	HTTPS

Security Group = stateful virtual firewall. Allow only needed traffic; restrict SSH to your IP. Public IP alone does not guarantee reachability.

3.4 User Data and Apache

Apache (httpd) serves web pages. Example for compatible Amazon Linux:

```
#!/bin/bash
dnf install -y httpd
systemctl enable --now httpd
echo "Hello codingwale $(hostname)" > /var/www/html/index.html
```

Allow HTTP/80, then open http://PUBLIC-IP. User Data usually runs at first boot; check the script and cloud-init logs if it fails.

```
sudo systemctl status httpd
sudo tail -80 /var/log/cloud-init-output.log
```

4. PROJECT — EC2 HELLO WORLD

Objective: Launch an EC2 server and display a webpage; connect using SSH/PuTTY or configure User Data.

Steps

• Launch EC2 with the correct AMI, instance type, key pair and VPC/subnet.

• Set Security Group inbound rules: SSH/22 from trusted IP; HTTP/80 from intended viewers.

• Connect with SSH or PuTTY (convert PEM to PPK with PuTTYgen if required).

• Install/start Apache and create /var/www/html/index.html, or use the User Data script at launch.

• Wait for instance checks; open the public IP in a browser.

• Confirm the Hello World text and hostname appear.

If it fails

Check instance status checks, public IP, subnet route, SSH username/key, Security Group, Apache status and /var/log/cloud-init-output.log.

Result: Hosted a simple Hello World webpage on EC2.

5. S3 — OBJECT STORAGE

5.1 Buckets, objects and keys

S3: Object storage. **Bucket:** container for objects. **Object:** file plus metadata. **Key:** unique object name, e.g. images/photo.jpg. A slash in a key is not a real filesystem folder.

5.2 Storage classes, versioning, lifecycle

Storage class: cost/access option. Standard = frequent access; Standard-IA = infrequent access; One Zone-IA = one AZ; Glacier classes = archival with different retrieval times/costs.

Versioning: retains versions of objects. **Lifecycle rule:** transitions or expires objects automatically.

5.3 Access and static website hosting

• IAM policies govern identities; bucket policies govern access to bucket/object resources.

• Block Public Access can prevent public policies/ACLs. Do not disable it unless public access is intended and understood.

• S3 website endpoint is for website hosting; object URLs are for direct object access. Website endpoints do not provide HTTPS by themselves.

• Public object access needs appropriate policy/configuration; uploading alone does not make an object public.

5.4 S3 website practical

Create bucket → Upload index.html + image → Enable versioning → Enable static website hosting → Configure access → Test website endpoint

Verification: HTML loads, image URL works, and a new upload creates a new version when versioning is enabled.

Result: Hosted a static webpage and displayed an image stored in S3.

7. VPC — NETWORKING (11 CORE TOPICS)

7.1 VPC (Virtual Private Cloud)

Meaning: Logically isolated network in AWS. **Why:** define IP range, organize resources, control connectivity/security. **Simple:** VPC = our AWS network.

7.2 Subnet

Meaning: IP range inside a VPC, in one AZ. Public subnet has a route to IGW; private subnet has no direct route to IGW. Public access also requires addressing and security configuration.

7.3 CIDR

Meaning: IP range notation. Example: VPC 10.0.0.0/16; subnet 10.0.1.0/24. Plan ranges without overlap; subnet CIDR must fit within VPC.

7.4 Internet Gateway (IGW)

Connects a VPC to the internet when routes, IP addressing and security rules are correct. Simple: IGW = internet connectivity for a VPC.

7.5 Route table

Rules decide where traffic goes. Example public route: 0.0.0.0/0 → IGW. A subnet must be associated with the route table for those routes to apply.

7.6 NAT Gateway

Lets private-subnet resources initiate outbound internet connections. Usually placed in a public subnet; private route points to NAT, and NAT subnet routes to IGW. It does not allow unsolicited inbound internet connections.

7.7 NACL

Subnet-level filter with inbound/outbound Allow and Deny rules. It is stateless, so return traffic must also be allowed.

7.8 Elastic IP

Static public IPv4 address that can be associated with supported AWS resources. Check charges and release unused Elastic IPs.

7.9 VPC Peering

Private connection between two VPCs. Requires non-overlapping CIDRs, routes on both sides and security rules. Peering is not transitive.

7.10 VPC Endpoint

Private path from a VPC to supported AWS services. Gateway endpoint (e.g. S3/DynamoDB) or interface endpoint (PrivateLink). Can reduce NAT usage for suitable traffic.

7.11 Default VPC

AWS-provided ready-to-use VPC in supported Regions, usually with default subnets and basic internet routing. Convenient for basic practice.

VPC architecture and flows

PUBLIC: Internet ↔ IGW ↔ Route table ↔ Public subnet → EC2

PRIVATE OUTBOUND: Private EC2 → Private route table → NAT Gateway → IGW → Internet

PRIVATE SERVICE ACCESS: Private EC2 → VPC Endpoint → Supported AWS service

Security Group = stateful resource-level filter (allow rules). NACL = stateless subnet-level filter (allow/deny). A public subnet is not automatically secure or internet-accessible.

8. DOCKER AND ECR

8.1 Docker basics

Dockerfile: build instructions. **Image:** packaged app/dependencies. **Container:** running image. **Registry:** stores images.

Dockerfile → Build image → Test container → Push image to registry

8.2 Amazon ECR

ECR (Elastic Container Registry) stores container images. Create repository → authenticate Docker → build → tag with repository URI → push → verify image.

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin ACCOUNT_ID.d
kr.ecr.us-east-1.amazonaws.com
docker build -t demo-app .
docker tag demo-app:latest ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/demo-repo:latest
docker push ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/demo-repo:latest
```

Replace ACCOUNT_ID and repository names. The identity needs ECR push permissions. Docker Hub is a general registry; ECR is AWS-managed.

9. ECS — CONTAINER ORCHESTRATION

9.1 Components

ECS: Runs/manages containers. **Cluster:** logical grouping. **Task definition:** blueprint (image, ports, CPU/memory, logs). **Task:** running instance. **Service:** maintains desired tasks. **Fargate:** runs containers without managing EC2 hosts.

9.2 Roles and networking

• Task execution role: lets ECS pull images and publish configured logs.

• Task role: grants the application permissions to call AWS APIs.

• Fargate tasks need appropriate subnets, route/connectivity, Security Groups and container port settings.

9.3 ECR → ECS Fargate project

Build image → Push to ECR → ECS cluster → Task definition → Service/tasks → Application endpoint

• Prepare app and Dockerfile; build/test image.

• Create ECR repository and push image.

• Create ECS cluster using Fargate.

• Create task definition with image URI, CPU/memory, container port, logs and roles.

• Create service; choose desired task count, VPC, subnets and Security Group.

• Allow intended inbound traffic to the task/container port.

• Wait for tasks to run; use public IP or configured load balancer/endpoint.

• Verify app in browser and inspect task logs.

Troubleshoot: stopped reason, image URI/tag, IAM roles, subnet internet/NAT/endpoints, port mapping, Security Groups and logs.

Result: Built an image, pushed it to ECR, deployed it on ECS Fargate and accessed the running app.

11. AWS LAMBDA

11.1 Components

Lambda: Runs code in response to events without managing servers. **Function:** code + config. **Runtime:** language environment. **Handler:** entry point. **Trigger:** event that invokes it. **Execution role:** permissions for what function code can access.

```
def lambda_handler(event, context):
    return {"statusCode": 200, "body": "Hello from Lambda!"}
```

11.2 Test and logs

Create function → select runtime/role → write handler → deploy → run test event → inspect response and CloudWatch Logs.

Invocation permission controls who can call the function; execution role controls what the function can do.

12. API GATEWAY

12.1 Core terms

API Gateway: creates/manages APIs. **Endpoint:** URL. **Route:** HTTP method + path (e.g. GET /hello). **Integration:** backend such as Lambda. **Stage:** deployed environment. **Authorization:** controls access.

12.2 API Gateway + Lambda workflow

Client → API Gateway route → Lambda integration → Response to client

- Create API and route/resource.
- Choose Lambda integration and configure required invoke permission.
- Configure authorization if required.
- Deploy to a stage when required by the API type.
- Call endpoint with browser/curl/API client.
- Verify response and Lambda logs.

If it fails: check URL/stage, route/method, auth, Lambda permission, integration settings and logs.

Result: Exposed backend functionality through an API endpoint. Keep API Gateway and basic Lambda test workflows separate if they were separate labs.

13. PROJECT — S3 + LAMBDA CSV CLEANER

Objective: On CSV upload, remove duplicate student records and save the cleaned file.

Upload to raw/ → S3 ObjectCreated event → Lambda reads CSV → Remove duplicates → Save to processed/

- Create S3 bucket in the required Region; use raw/ and processed/ prefixes.
- Create Lambda function with CSV-cleaning logic and an execution role.
- Grant only required S3 read access to raw/ and write access to processed/; allow CloudWatch logging.
- Configure S3 ObjectCreated trigger limited to raw/.
- Upload a test CSV with duplicate rows.
- Lambda reads the event's bucket/key, processes the file and writes cleaned output to processed/.
- Verify output rows/file and inspect CloudWatch Logs.

Prevent recursive triggers: trigger only on raw/ and write output to processed/. Check Region, permissions, object key, CSV format, timeout and logs.

Result: Automated CSV deduplication using S3 events and Lambda.

14. ELASTIC LOAD BALANCING

14.1 Types

Type	Purpose
ALB	HTTP/HTTPS application traffic (Layer 7)
NLB	High-performance Layer 4 TCP/UDP/TLS use cases
GWLB	Deploy/scale network virtual appliances
CLB	Legacy load balancer

14.2 ALB components

Listener: checks protocol/port. **Rule:** matches requests and routes them. **Target group:** registered targets. **Health check:** checks target health. **DNS name:** client endpoint.

Client → ALB listener → Rule → Target group → Healthy target

14.3 Three-EC2 ALB project

• Launch three EC2 instances in the intended VPC/subnets.

• Install Apache; serve a page that shows each instance hostname.

• Create target group with correct target type, protocol, port and health-check path.

• Register the three instances.

• Create ALB and listener; forward requests to the target group.

• Allow application traffic to ALB; allow EC2 traffic only from the ALB Security Group where possible.

• Wait for all targets to become healthy.

• Open ALB DNS and refresh/repeat requests to observe responses.

Troubleshoot: Apache service, target port/path, target health, Security Groups, routes and instance startup logs.

Result: ALB distributed web requests among three EC2 web servers. Responses may vary; strict round-robin is not guaranteed.

16. AUTO SCALING GROUP

16.1 Components

ASG: Maintains desired EC2 capacity and can scale based on policies. **Launch template:** AMI, instance type, key, Security Groups, User Data and other settings.

Minimum: lowest capacity. **Desired:** target count. **Maximum:** upper limit. **Scaling policy:** adjusts desired capacity. **Health check:** helps replace unhealthy instances.

16.2 ASG + ALB project

• Create launch template with AMI, instance type, Security Group and User Data.

• Create ASG; choose VPC/subnets and min, desired and max capacity.

• Attach the ALB target group.

• Configure EC2/ELB health checks and scaling policy as required.

• Wait for instances to launch and register healthy.

• Test the ALB DNS and verify desired capacity/target health.

If the ASG manages instances, it may replace manually terminated instances. Troubleshoot launch template, permissions, subnet capacity, User Data and health checks.

Result: ASG maintained EC2 capacity while ALB distributed traffic.

18. CLOUDWATCH AND SNS

18.1 CloudWatch

Metrics: numbers over time. **Logs:** records. **Alarms:** evaluate metrics against conditions. **Dashboards:** combine charts and metrics.

OK: not breaching. **ALARM:** condition met. **INSUFFICIENT_DATA:** not enough data to evaluate.

18.2 SNS

SNS: notification service. **Topic:** message channel. **Subscription:** endpoint attached to topic. Email subscriptions must be confirmed.

18.3 CPU alarm + email project

• Open CloudWatch → EC2 CPUUtilization metric.

• Create alarm; set threshold CPU > 30%.

• Set metric period, evaluation periods and missing-data treatment.

• Choose/create SNS topic and add email subscription.

• Confirm subscription from the email inbox.

• Set the alarm action to notify the SNS topic.

• Create alarm and verify its configuration/state.

• If testing, generate CPU load safely and long enough to meet evaluation periods; verify email.

EC2 CPU metric → CloudWatch alarm → SNS topic → Confirmed email subscription → Notification

A threshold alone does not mean an instant alert; the metric must breach the configured evaluation condition. Evidence: alarm configuration + received email.

20. SYLLABUS EXTRAS — THEORY / FUTURE

20.1 EKS — Elastic Kubernetes Service

Managed Kubernetes service. AWS manages the control plane; worker nodes or Fargate run workloads. Flow: create cluster → configure access → deploy Kubernetes resources → expose service → monitor.

EKS is different from Kops: EKS is an AWS-managed Kubernetes service; Kops is a tool used to provision/manage Kubernetes clusters.

20.2 AWS-native CI/CD

CI: integrate code changes and build/test. **CD:** prepare or deploy validated changes.

Service	Role
CodeBuild	Build and test code
CodePipeline	Coordinate pipeline stages
CodeDeploy	Automate supported deployments

Source repository → Build/test → Artifact/image → Deploy → Verify

Keep this as theory unless an AWS-native pipeline was actually built. Jenkins/GitHub projects are separate tools/workflows.

21. TROUBLESHOOTING, CLEANUP & REVISION

21.1 Quick troubleshooting map

Problem	Check first
Cannot SSH	Instance running, correct key/user, port 22 source rule, public route/IP
Website won't load	Apache status, port 80, public IP/DNS, routes, Security Groups
User Data failed	Script syntax, package manager, cloud-init-output.log
S3 object denied	IAM/bucket policy, Block Public Access, object key
ECS task stopped	Stopped reason, image URI, task/execution roles, networking, ports, logs
Lambda failed	Handler/runtime, trigger event, execution role, timeout, CloudWatch Logs
ALB target unhealthy	Target port, health path, service running, SG rules
No SNS email	Subscription confirmed, alarm action/topic, threshold/evaluation periods

21.2 Cost cleanup checklist

• Delete unused EC2 instances and unused EBS volumes/snapshots where safe.

• Delete unused ALBs/target groups and Auto Scaling Groups after confirming dependencies.

• Delete unused ECS services/tasks/clusters and ECR images/repositories if no longer needed.

• Check NAT Gateways and release unused Elastic IPs.

• Check S3 objects and retained versions before deleting buckets.

• Review unused CloudWatch log groups, Lambda resources and other billable resources.

• Check Billing/Cost Explorer after labs.

Stopping EC2 does not stop every charge. EBS, NAT Gateways, load balancers, storage and other provisioned resources may still cost money.

21.3 Final workflow revision

Lab	Remember this flow
EC2 Hello World	EC2 → SSH/PuTTY or User Data → Apache → Browser
S3 website	Bucket → HTML/image → Versioning/hosting/access → Test
Container deployment	Docker → ECR → ECS Fargate → App
Lambda/API	Request/event → API Gateway or trigger → Lambda → Response/logs
CSV cleaner	S3 raw/ → Lambda → Deduplicate → processed/
Load balancing	3 EC2 → Target group → ALB → DNS
Auto Scaling	Launch template → ASG → EC2 → Target group/ALB
Monitoring	CPU metric → CloudWatch alarm → SNS → Email

21.4 Essential rules

• IAM controls permissions; always use least privilege.

• Security Groups are stateful; NACLs are stateless.

• S3 object upload does not automatically make it public.

• ECR stores images; ECS runs/manages containers.

- Lambda execution role and invocation permission are different.
- ALB sends traffic to healthy registered targets.
- ASG maintains desired capacity; it can replace instances.
- CloudWatch monitors; SNS delivers notifications.
- Always verify the result and clean up unused resources.